ISCD Homework 4

Anurag Bhunia(bmat2508@isibang.ac.in)

Question 1

a)

```r
set.seed(2026)
#Generating p
p=runif(1)
#Getting 100 Uniform(0,1) Samples
u=runif(100)
#Appling the inverse CDF transform
x=ceiling(log(1-u)/log(1-p))
x
```

```
##   [1] 1 1 1 1 1 1 2 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 3 1 2 1 1 1 1 1
##  [38] 2 1 1 2 1 1 1 1 3 1 1 1 3 1 3 2 1 2 2 1 3 1 1 2 1 1 1 1 1 1 1 1 2 1 2 2 1
##  [75] 1 2 1 1 2 1 3 2 3 1 4 1 3 1 1 3 2 2 1 1 1 2 1 1 1 1
```

```r
set.seed(2025)
#Setting r=5
r=5
nb=numeric(100)
for(i in 1:100){
    U=runif(r)
    X=ceiling(log(1-U)/log(1-p))
    nb[i]=sum(X)
}
nb
```

```
##   [1]  7  6  7  7  7  9  5 10 14  7  6  6  8  7  9  7  8  5  9  7  8  9  8  6  7
##  [26] 10  5  8  9  6 10  6  7  6  7  6 10  8  6  7  7  6 10  8  5  5 10  7  6  5
##  [51]  8  5  6  5  7  9  7  7  7  8  5  5  8  8  9  9  5 10  9  9  8  6  9  6  8
##  [76] 10  7  5  9  5  6  5  6  8  8  6  8  7  5  6  9  6  7  5  6  6  7  8  6  7
```

b)

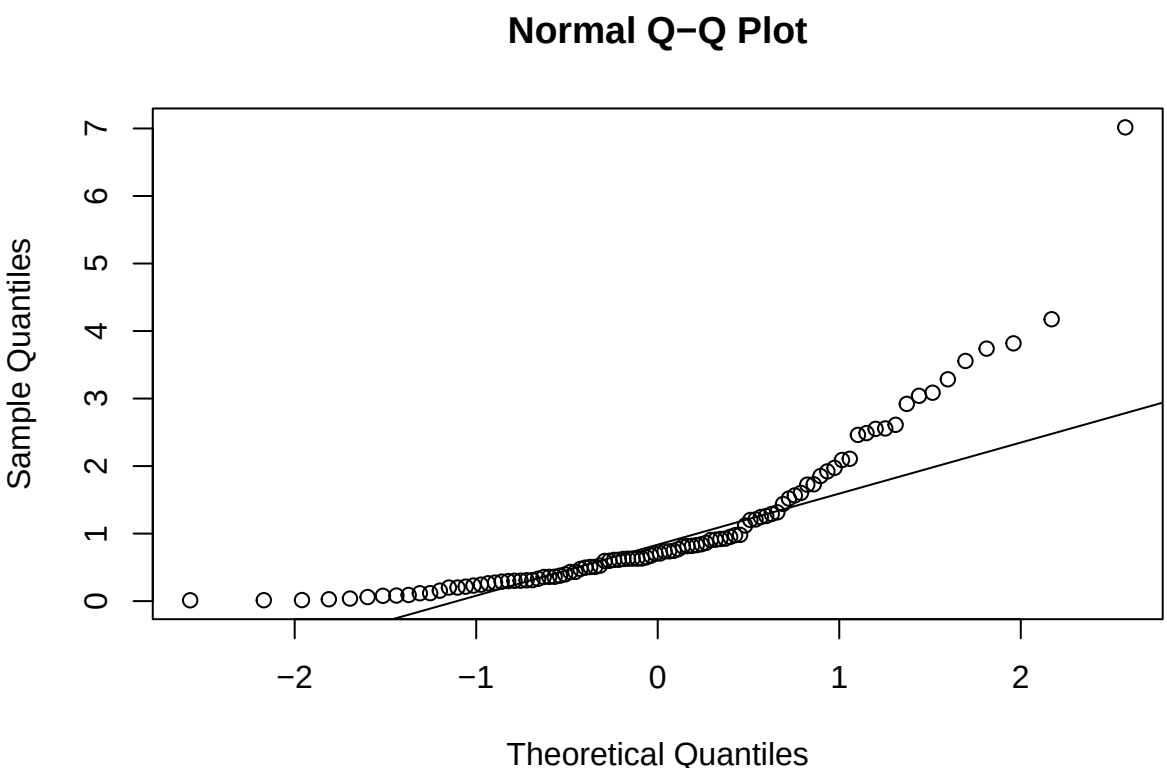
```r
set.seed(2023)
u=runif(100)
#Generating 100 Cauchy samples
c=tan(pi*(u-0.5))
c
```

```
##   [1]  -0.10527170  -0.56959513  -1.78145729  -0.33845014 -10.44172337
##   [6]  -2.50534116  -0.23620905   0.38816398  -0.92027258  -0.07451823
##  [11]   2.16592747  -1.97982203  -1.57105188 438.02753453   1.84279952
##  [16]  -2.07999860  -0.53008274   2.76738545  -0.63528804   0.77073919
##  [21]   0.42586195  -0.19671676  -0.55018009   4.68719563   0.79065383
##  [26]   0.43591973   0.97793635  -2.73839498  -0.86342258  -3.12714627
##  [31]   1.04846458   0.56454260   0.85061529  -0.12969318   0.61020485
```

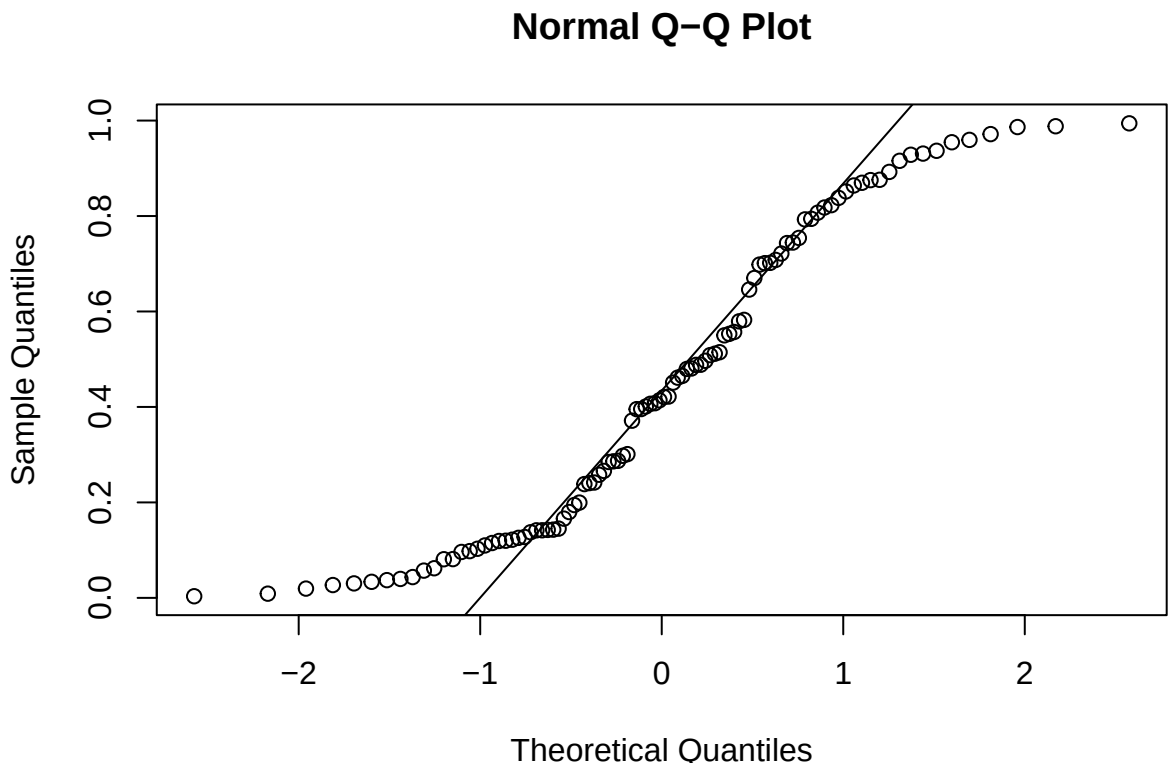
```
## [36]  0.72901074  1.52310746  0.68503909  0.80846198  2.79356464
## [41] -0.54854071  0.58532047 -0.38054926 -1.09819966  2.73698896
## [46]  1.38966549  0.31255686 13.82616854 -0.59946248  1.42456805
## [51] -9.81624094 -0.16389646 -0.91147185 -0.29302392 -1.42669412
## [56]  0.13747646  4.83675213  1.16370660 102.12552213  1.60485918
## [61] -0.35398183 -0.37697310  3.04955231 -0.63968545 -1.30021905
## [66]  0.20439563 -0.05634291 -4.82007512 -0.93153915 -0.58997599
## [71] -0.58900875 -3.34939505  0.69809849 -2.60942611  2.32281274
## [76] -2.33760082  1.75803393 -1.08048639 -0.15703664  1.74424500
## [81]  0.57371294  0.13949897  0.55916854  2.38265455  0.58458353
## [86] -1.25996189 -0.25985842 -0.11747884  0.39160513 -0.05117026
## [91]  1.04019953 -1.19640482 36.07798358 15.68191428  2.10840224
## [96] -1.05479146  0.43771805  0.96327045  1.16641774  0.61209466
```

2.

```r
set.seed(2024)
ex=rexp(100)
u=runif(100)
#Deviation of the exponential distribution
qqnorm(ex)
qqline(ex)
```




```r
#Deviation of the uniform distribution
qqnorm(u)
qqline(u)
```



3.

0) This is known as the method of moments. Given a sample, we compute its mean and variance. Knowing the distribution, we equate both to their theoretical formulas (in this case, a binomial distribution). The expressions for p-hat and n-hat are exactly those which one would get after solving for the theoretical values of p and n respectively. This is used to estimate the parameters pertaining to the given sample.

a)

```
n=100
N=20
p=0.4
x=rbinom(n,N,p)
m1=mean(x)
m2=mean(x^2)
Nhat=m1^2/(m1-(m2-m1^2))
phat=(m1-(m2-m1^2))/m1
#Estimated sample size
Nhat
```

```
## [1] 22.56826
```

```
#Estimated probability
phat
```

```
## [1] 0.3784075
```

b) For n=1000, we have

```
n=1000
N=20
p=0.4
x=rbinom(n,N,p)
```

```r
m1=mean(x)
m2=mean(x^2)
Nhat=m1^2/(m1-(m2-m1^2))
phat=(m1-(m2-m1^2))/m1
#Estimated sample size
Nhat
```

```
## [1] 17.11058
```

```r
#Estimated probability
phat
```

```
## [1] 0.4685405
```

The values are quite different from when we had just 100 samples.

c)

```r
#Bin(10,0.1)
n=100
N=10
p=0.1
x=rbinom(n,N,p)
m1=mean(x)
m2=mean(x^2)
Nhat=m1^2/(m1-(m2-m1^2))
phat=(m1-(m2-m1^2))/m1
#Estimated sample size
Nhat
```

```
## [1] 7.148867
```

```r
#Estimated probability
phat
```

```
## [1] 0.1314894
```

```r
n=1000
N=10
p=0.1
x=rbinom(n,N,p)
m1=mean(x)
m2=mean(x^2)
Nhat=m1^2/(m1-(m2-m1^2))
phat=(m1-(m2-m1^2))/m1
#Estimated sample size
Nhat
```

```
## [1] 11.22309
```

```r
#Estimated probability
phat
```

```
## [1] 0.0885674
```

```r
#Bin(10,0.9)
n=100
N=10
p=0.9
x=rbinom(n,N,p)
```

```r
m1=mean(x)
m2=mean(x^2)
Nhat=m1^2/(m1-(m2-m1^2))
phat=(m1-(m2-m1^2))/m1
#Estimated sample size
Nhat
```

```
## [1] 10.12208
```

```r
#Estimated probability
phat
```

```
## [1] 0.8901332
```

```r
n=1000
N=10
p=0.9
x=rbinom(n,N,p)
m1=mean(x)
m2=mean(x^2)
Nhat=m1^2/(m1-(m2-m1^2))
phat=(m1-(m2-m1^2))/m1
#Estimated sample size
Nhat
```

```
## [1] 9.943295
```

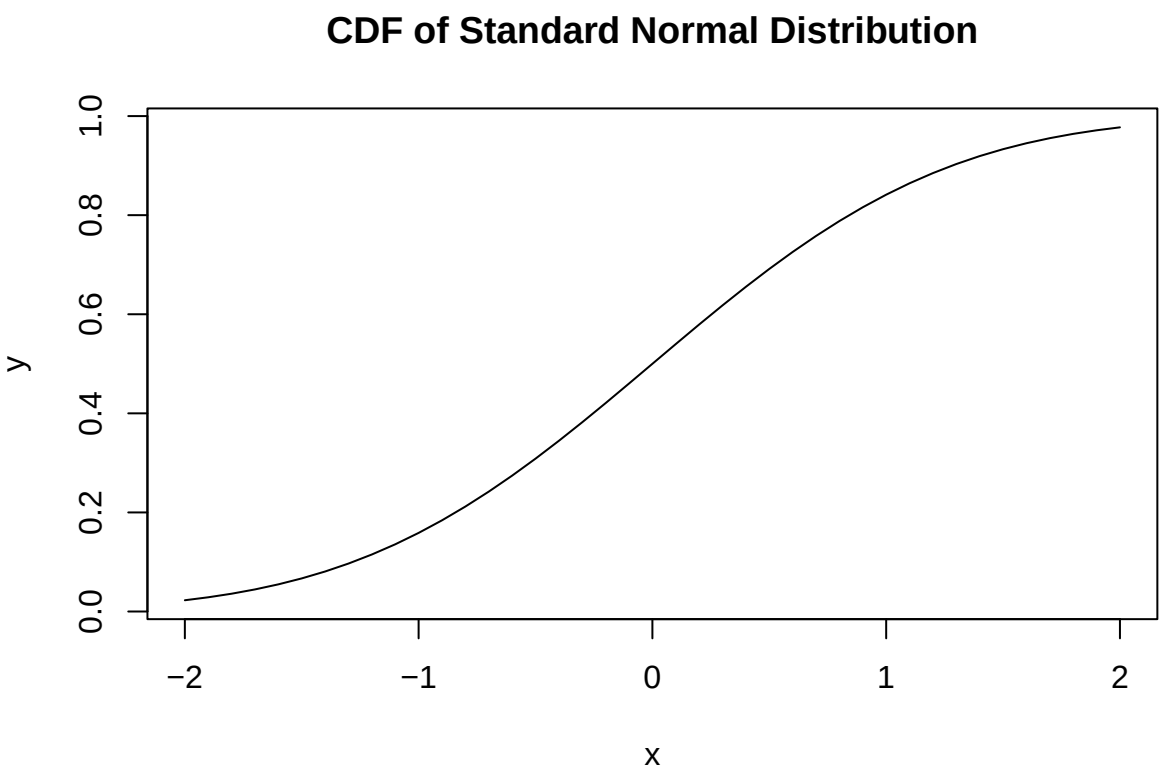
```r
#Estimated probability
phat
```

```
## [1] 0.9041268
```

4.

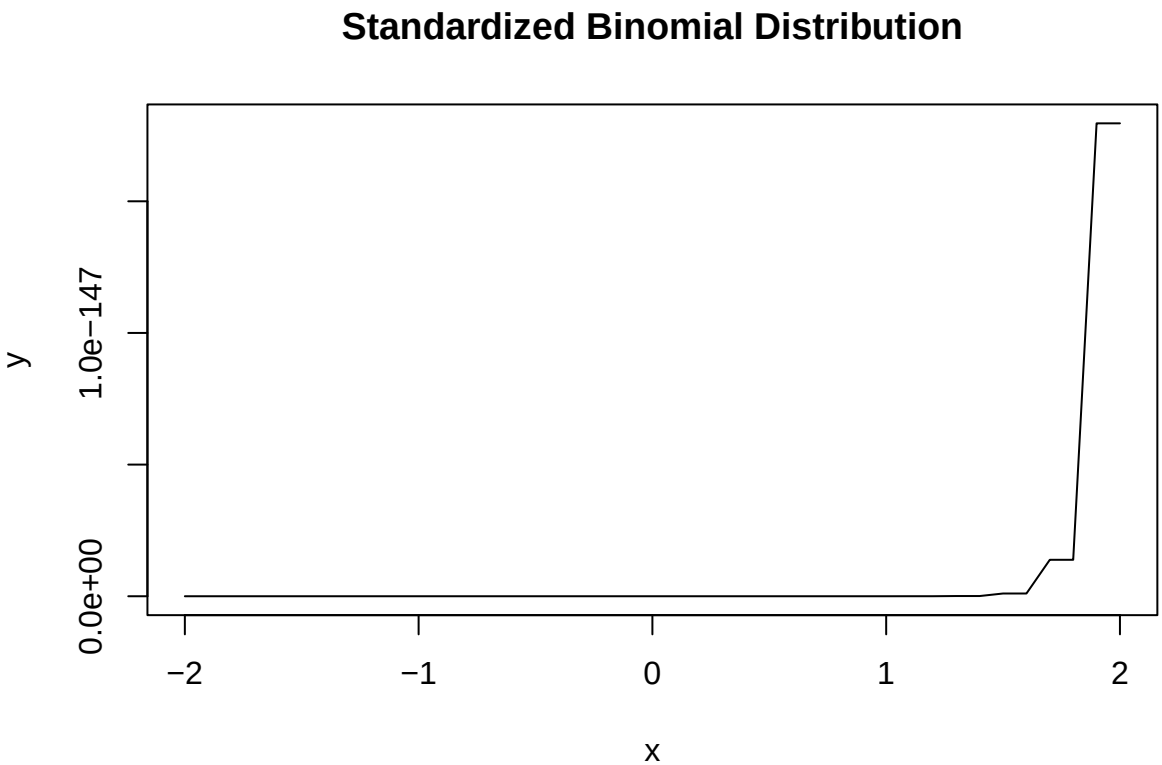
a)

```r
x=seq(-2,2,by=0.1)
y=pnorm(x,0,1)
plot(x,y,type="l",main="CDF of Standard Normal Distribution")
```



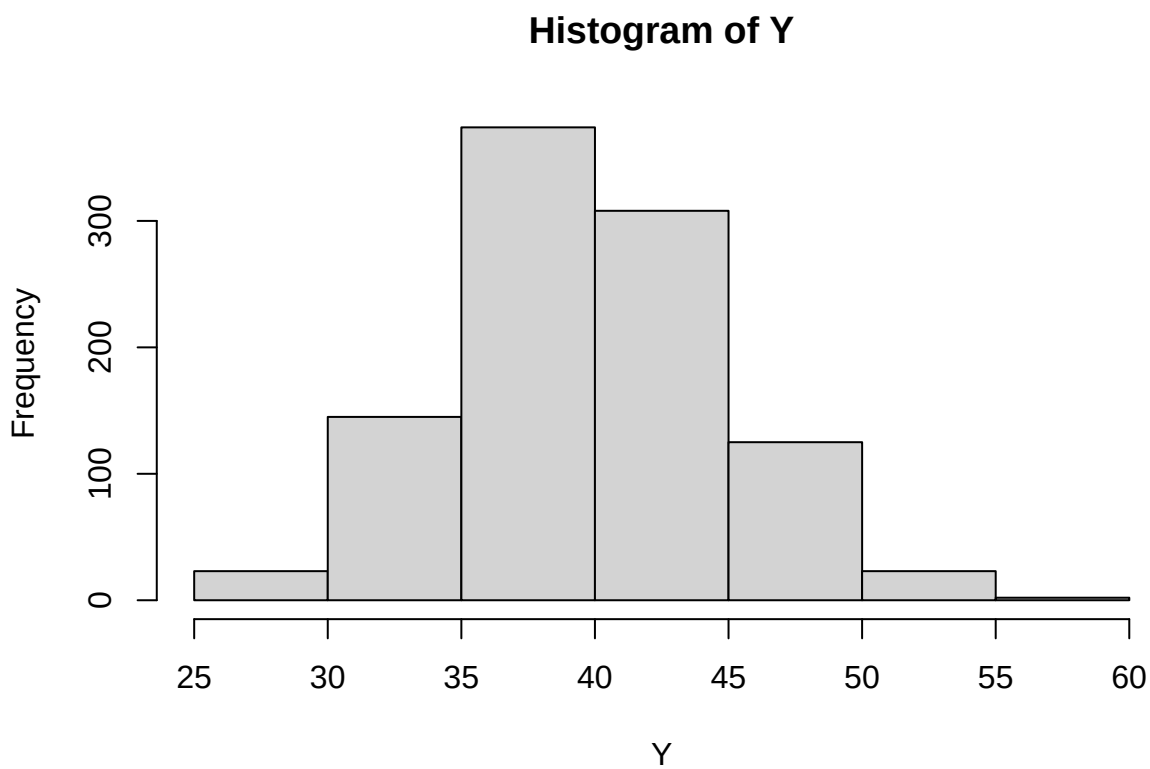
b)

```
m=100
p=0.4
x=seq(-2,2,by=0.1)
mu=m*p
sd=sqrt(m*p*(1-p))
c=x*sd+mu
y=pbinom(c,n,p)
plot(x,y,type="l",main="Standardized Binomial Distribution")
```

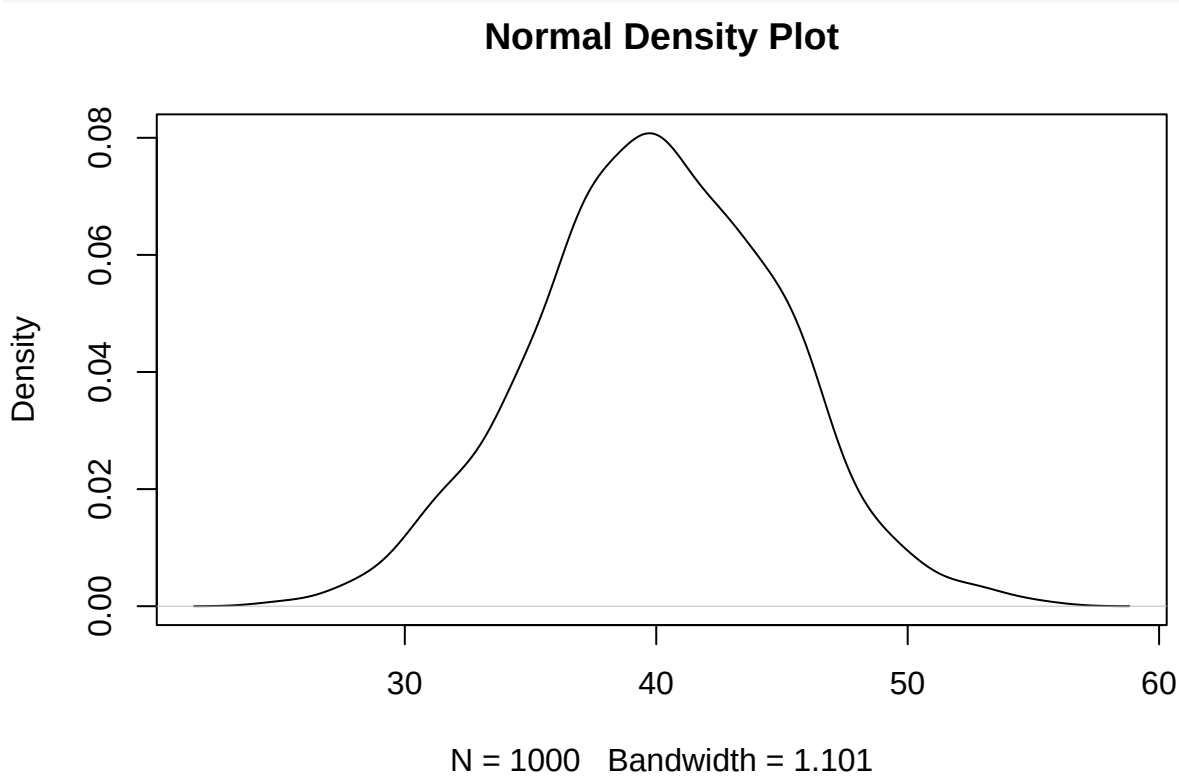


c)

```
set.seed(1011)
m=100
p=0.4
Y=rbinom(1000,m,p)
#Histogram for Binomial Distribution
hist(Y)
```




```
mu=m*p
sd=sqrt(m*p*(1-p))
Z=rnorm(1000,mu,sd)
d=density(Z)
plot(d,type="l",main="Normal Density Plot")
```



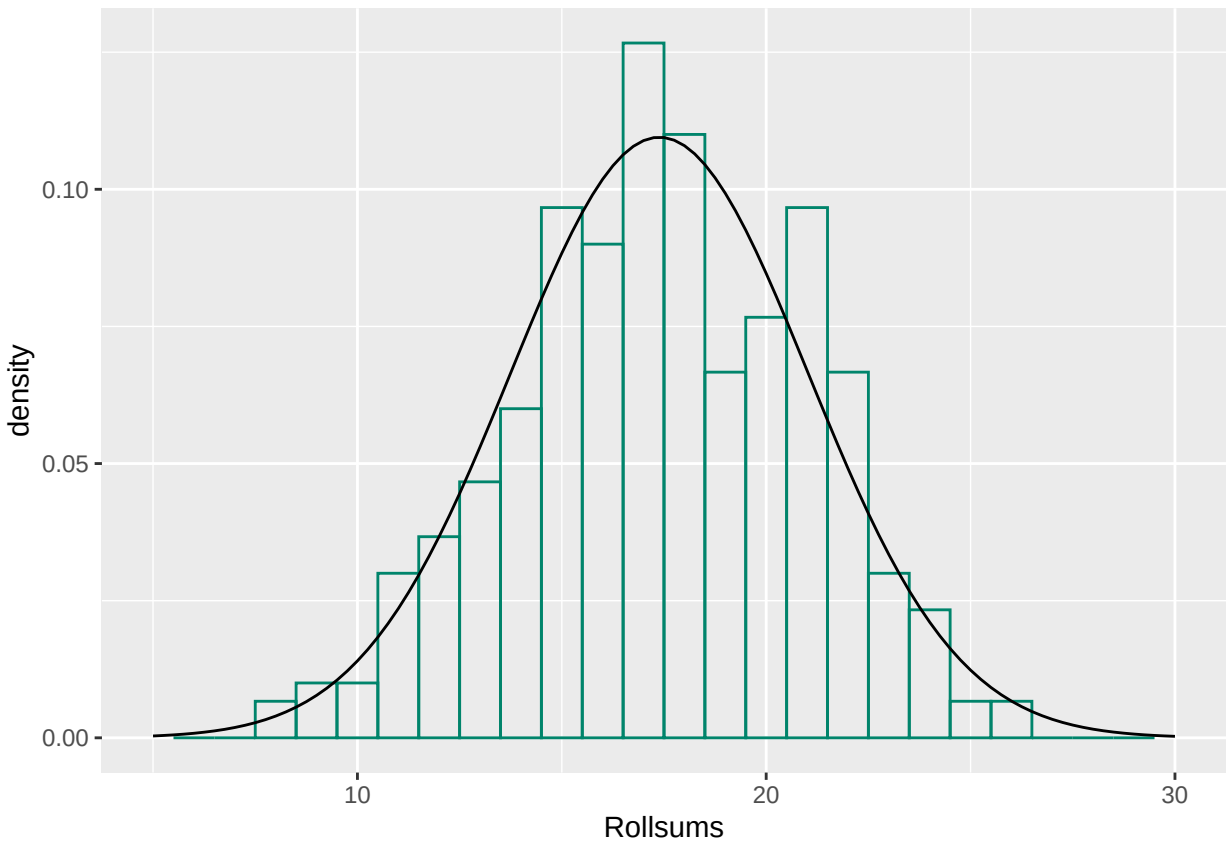
From the 2 plots, one can conclude that the binomial histogram has a similarity in shape with the normal density plot. This is a consequence of the Central Limit Theorem, as the sample size increases, the binomial distribution ressembles the normal.

5.

```r
# Define the possible outcomes of a single die roll (1 through 6)
x = c(1,2,3,4,5,6)

# Define the probability for each outcome (1/6 for a fair die)
probx = c(1/6, 1/6, 1/6, 1/6, 1/6, 1/6)

# Simulate 1500 independent die rolls using the defined probabilities
Rolls = sample(x, size = 1500, replace = T, prob = probx)

# Organize the 1500 rolls into a matrix with 5 rows
# This effectively creates 300 groups, where each group has 5 die rolls
Rollm = matrix(Rolls, nrow = 5)

# Calculate the sum of each column (the sum of the 5 rolls in each group)
Rollsums = apply(Rollm, 2, sum)

# Load the ggplot2 library for advanced data visualization
library(ggplot2)

# Manually define the Normal density function formula:
# 1/(sqrt(2*pi)*s) * exp(-(x-a)^2 / (2*s^2))
density = function(x, a, s) { (1/((2*pi)^(0.5)*s )) * exp(-(x-a)^2/(2*s^2)) }

# Convert the vector of sums into a data frame format required by ggplot2
dfrolls = data.frame(Rollsums)

# Calculate the sample mean (mu) and standard deviation (sigma) of the sums
mu = mean(dfrolls$Rollsums)
sigma = sd(dfrolls$Rollsums)

# Create the visualization
ggplot(data = dfrolls) +
  # Draw a histogram of the sums; y=..density.. scales it to match a probability curve
  geom_histogram(mapping = aes(x = Rollsums, y = ..density..), color = "#00846b", fill = NA, binwidth =
  # Set the x-axis limits to focus on the range of possible sums (5 to 30)
  xlim(5,30) +
  # Overlay the manually defined Normal density function using the calculated mu and sigma
  geom_function(fun = density, args = list(a = mu, s = sigma), color = "black")
```

```
## Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
## i Please use `after_stat(density)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_bar()`).
```



This is a demonstration of the Central Limit Theorem. It simulates rolling a fair die 1500 and recording sums of die rolls from sets of 5. The sum is distributed almost normally as its corresponding histogram aligns closely with the normal density curve.